MONITORING AND LOGGING

© 2011-25 National University of Singapore. All Rights Reserved.

Topics

- Introduction to MLOps
- Pipeline design
- **Monitoring and Logging**
- MLOps with Cloud platforms

© 2011-24 National University of Singapore. All Rights Reserved.

Monitoring and Logging

Infrastructure Metric

- CPU & memory usage
- Request volume – IOPS
- Application availability
- Peak usage time
- Average response time - Outliers and percentiles
- Request success & failure rate
- Request and Response payload size
- Garbage collection time

© 2011-24 National University of Singapore. All Rights Reserved.

Metric on Scalability and Availability

- Uptime – (MTBF-MTTR)/MTBF
  - Mean time between failures & Mean time to recover
  - Measured in nines
  - 5 nines – 5 minutes downtime per year, 4 nines – 50 minutes downtime per year, 3 nines – 9 hours downtime per year
- Yield – Successful requests/Total requests
  - Yield is more important than uptime – we could bring down the system when there are no requests
- Harvest – Data available/Total data
  - How many servers that has the data are available?

© 2011-24 National University of Singapore. All Rights Reserved.

Metric Tools Option

- Prometheus
  - Time-series database that scrapes metrics from your application (usually via HTTP)
- Grafana
  - Visualization tool that connects to Prometheus (or other data sources) to build dashboards
- Any other Cloud platform tool like AWS CloudWatch, AWS X-Ray

© 2011-24 National University of Singapore. All Rights Reserved.

Metric for ML Models – Classification Models

Metric	Description
Accuracy	(TP + TN) / Total — % of correct predictions
Precision	TP / (TP + FP) — how many predicted positives are correct
Recall (Sensitivity)	TP / (TP + FN) — how many actual positives are captured
F1 Score	Harmonic mean of precision and recall F1=2×(Precision+Recall/Precision×Recall)

© 2011-24 National University of Singapore. All Rights Reserved.

Metric for ML Models – Regression Models

Metric	Description
Mean Absolute Error (MAE)	Average absolute error
Mean Squared Error (MSE)	Average squared error
Root Mean Squared Error (RMSE)	Square root of MSE
R^2 (Coefficient of Determination)	% of variance explained by model

© 2011-24 National University of Singapore. All Rights Reserved.

Latency and Throughput

Metric	Description
Inference Latency	Time taken to make a single prediction
Throughput	Predictions per second
Time to Load Model	Startup latency during deployment

© 2011-24 National University of Singapore. All Rights Reserved.

Data Drift & Quality Metrics

Metric	Description
Data Drift	Distribution shift in **input features** (e.g. KS test, PSI)
Target Drift	Change in **label distribution** over time
Feature Missing Rate	% of missing values in inputs
Outlier Rate	How often the data receives are significantly different during training dataset.
Feature Importance Drift	Change in key contributing features
Input Schema Changes	Structural difference in incoming data

© 2011-24 National University of Singapore. All Rights Reserved.

Monitoring and Logging

ML Monitoring Tools Option

- Evidently AI
  - Open-source and integrates well with pipelines
- AWS SageMaker Model Monitor
  - Data & model drift, quality monitoring, alerting
  - Native to AWS SageMaker
- Azure ML Data Monitor
  - Retraining triggers
  - Native to Azure ML
- Google Vertex AI Model Monitoring

© 2011-24 National University of Singapore. All Rights Reserved.

Monitoring and Logging

- Large part of an engineer's life is spent on debugging, analysis and performance tuning

Common information:

- Experiment run and related
- Hyperparameters
- Model registry information
- Metrics (Accuracy, drift etc)

© 2011-24 National University of Singapore. All Rights Reserved.

Monitoring and Logging

Logging problem in Real world

- Real world there will be may agents and models hosted in various dockers
- Each agent or model may have their own logging

Solution

- Aggregated Logging

© 2011-24 National University of Singapore. All Rights Reserved.

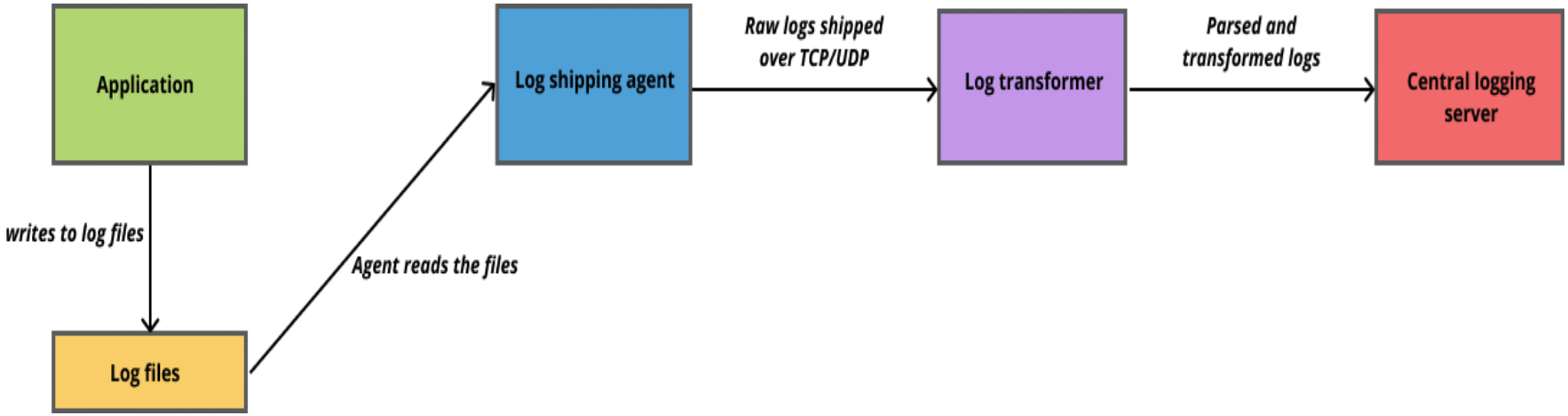
Monitoring and Logging

What is aggregated logging ?

- Application writes logs to local file
- A lightweight log shipping agent ships raw logs to a remote Log parser/transformer
- The heavy log transformer stores logs to the central log server

© 2011-24 National University of Singapore. All Rights Reserved.

What is aggregated logging ?



© 2011-24 National University of Singapore. All Rights Reserved.

Logging Tools Option

- ElasticSearch Logstash Kibana (ELK)
  - ElasticSearch – centralized logging server
  - Logstash – Log transformer
  - Kibana – User interface
- AWS tools
  - AWS CloudWatch
  - AWS OpenSearch (Fully managed ELK)

© 2011-24 National University of Singapore. All Rights Reserved.

Logging Tools Option

- Azure tools
  - Azure Monitor
  - Azure Log Analytics
- Google tools
  - Cloud Logging
  - Cloud Monitoring
  - Cloud Trace

© 2011-24 National University of Singapore. All Rights Reserved.

Summary.

- General infrastructure metrics
- Metrics on availability and scalability
- Different types of ML Metrics
- Aggregated Logging

© 2011-24 National University of Singapore. All Rights Reserved.

APPENDIX

An example of Model Monitoring using AWS SageMaker

© 2011-25 National University of Singapore. All Rights Reserved.

Jupyter Notebook

- Login to AWS Console Service
- Go to AWS SageMaker AI service
- Create an instance of Jupyter Notebook

The next few slides are the sample codes for the Notebook.

© 2011-24 National University of Singapore. All Rights Reserved.

Environment Setup

Install the required libraries

- pip install sagemaker scikit-learn boto3 –upgrade
- sagemaker : AWS Sagemaker SDK
- boto3 : AWS SDK for Python

© 2011-24 National University of Singapore. All Rights Reserved.

Environment Setup

```
import numpy as np
import pickle
import boto3
import sagemaker
from sagemaker.sklearn.model import
SKLearnModel
```

© 2011-24 National University of Singapore. All Rights Reserved.

Training the data

```
# Generate dummy training data
X_train = np.random.rand(100, 3)
y_train = X_train @ np.array([0.5, 1.2, -0.3]) + np.random.normal(0, 0.1, 100)
# Train a simple linear regression model
from sklearn.linear_model import LinearRegression
import pickle
model = LinearRegression()
model.fit(X_train, y_train)
# Save the trained model
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)
print("Model trained and saved as model.pkl")
```

© 2011-24 National University of Singapore. All Rights Reserved.

Upload Model to Amazon S3

```
# Initialize session
sagemaker_session = sagemaker.Session()
bucket = sagemaker_session.default_bucket()
# Define model path
model_s3_path = f"s3://{bucket}/sagemaker-model/model.pkl"
# Upload model.pkl to S3
s3 = boto3.client("s3")
s3.upload_file("model.pkl", bucket, "sagemaker-
model/model.pkl")
print(f"Model uploaded to {model_s3_path}")
```

© 2011-24 National University of Singapore. All Rights Reserved.

Compress and upload the Model

Compress the model

```
!tar -czvf model.tar.gz model.pkl
```

Upload the compressed model

```
s3.upload_file("model.tar.gz", bucket, "sagemaker-model/model.tar.gz")

print(f"Model uploaded to: s3://{bucket}/sagemaker-model/model.tar.gz")
```

© 2011-24 National University of Singapore. All Rights Reserved.

Create an inference script

```
%%writefile inference.py
import pickle
import numpy as np
def model_fn(model_dir):
    with open(f"{model_dir}/model.pkl", "rb") as f:
        model = pickle.load(f)
    return model
def predict_fn(input_data, model):
    return model.predict(input_data)
```

© 2011-24 National University of Singapore. All Rights Reserved.

Create a SageMaker Model

```
role = sagemaker.get_execution_role()
model_s3_path = f"s3://{bucket}/sagemaker-model/model.tar.gz"
sklearn_model = SKLearnModel(
    model_data=model_s3_path,
    role=role,
    entry_point="inference.py",
    framework_version="0.23-1"
)
print("SageMaker model created successfully!")
```

© 2011-24 National University of Singapore. All Rights Reserved.

Deploy Model to a SageMaker Real-Time Endpoint

```
predictor = sklearn_model.deploy(
    instance_type="ml.m5.large",
    initial_instance_count=1
)
print(f"Model deployed at endpoint: {predictor.endpoint_name}")
```

© 2011-24 National University of Singapore. All Rights Reserved.

Perform Some Sample Real-Time Inference

```
test_data = np.array([[1.5, 2.0, -1.0]])
response =
predictor.predict(test_data.tolist())
print("Prediction:", response)
```

© 2011-24 National University of Singapore. All Rights Reserved.

Enable Model Monitor

```
from sagemaker.model_monitor import DataCaptureConfig
# Enable data capture while deploying
data_capture_config = DataCaptureConfig(
    enable_capture=True,
    sampling_percentage=100,
    destination_s3_uri=f"s3://{bucket}/sagemaker-model-monitor/data-capture"
)
# Redeploy with data capture enabled
predictor = sklearn_model.deploy(
    instance_type="ml.m5.large",
    initial_instance_count=1,
    data_capture_config=data_capture_config
)
print(f"Data capture enabled at: s3://{bucket}/sagemaker-model-monitor/data-capture")
```

© 2011-24 National University of Singapore. All Rights Reserved.

Enable Data Capture

- ***DataCaptureConfig***, which is SageMaker's helper class to configure logging of inputs/outputs of your model in real time.
- ***enable_capture* = True** → turns on the data capture feature.
- ***sampling_percentage* = 100** → means capture 100% of inference requests (you could set 10% or 50% if you only want a sample).
- **destination_s3_uri=...** → defines where in S3 the captured data will be stored.
- **In this case:**
  - Bucket: whatever is in your bucket variable
  - Folder: sagemaker-model-monitor/data-capture
- Captured data will be saved as JSON lines files in S3.

© 2011-24 National University of Singapore. All Rights Reserved.

Redeploying the Model with Data Capture

- ***sklearn_model.deploy(...)*** → deploys your scikit-learn model as a real-time endpoint.
- ***instance_type***="ml.m5.large" → chooses the compute instance type for the endpoint.
- ***initial_instance_count***=1 → runs 1 instance of the endpoint.
- ***data_capture_config***=data_capture_config → enables logging using the configuration you set earlier.
- This means every request/response will be captured and sent to the S3 bucket.

© 2011-24 National University of Singapore. All Rights Reserved.

Import and Setup the Monitor

```
from sagemaker.model_monitor import DefaultModelMonitor
monitor = DefaultModelMonitor(
    role=role,
    instance_count=1,
    instance_type='ml.m5.large',
    volume_size_in_gb=20,
    max_runtime_in_seconds=3600,
)
```

© 2011-24 National University of Singapore. All Rights Reserved.

Import and Setup the Monitor

- **DefaultModelMonitor** → A prebuilt SageMaker class that automates monitoring jobs.

- Parameters:

  - ***role***=role → IAM role with permissions to read/write from S3 and run monitoring jobs.

  - ***instance_count***=1 → Use 1 instance for monitoring.

  - ***instance_type***='ml.m5.large' → The compute resource to run the job.

  - ***volume_size_in_gb***=20 → The storage attached to the monitoring instance.

  - ***max_runtime_in_seconds***=3600 → Each monitoring job has at most 1 hour to complete.

© 2011-24 National University of Singapore. All Rights Reserved.

Set Up a Monitoring Schedule

```
monitor.create_monitoring_schedule(
    monitor_schedule_name="model-monitor-schedule",
    endpoint_input=predictor.endpoint_name,
    output_s3_uri=f"s3://{bucket}/sagemaker-model-monitor/reports",
    statistics=None,
    constraints=None,
    schedule_cron_expression="cron(0 * ? * * *)",  # Run every hour
)
print("Monitoring schedule created!")
```

© 2011-24 National University of Singapore. All Rights Reserved.

Setup a Monitoring Schedule

- ***monitor_schedule_name*** → A friendly name for your monitoring schedule.

- ***endpoint_input***=predictor.endpoint_name → Tells the monitor which deployed model endpoint to watch.

- ***output_s3_uri*** → Where the monitoring **reports and metrics** will be stored in S3.

- ***statistics***=None, ***constraints***=None → If you had a **baseline** (reference dataset), SageMaker would compare incoming data against it for **data drift / quality issues**. Here you haven't provided one (so it just collects data).

- ***schedule_cron_expression***="cron(0 * ? * * *)" → This is an AWS cron expression. It means run at minute 0 of every hour → so every hour the monitor runs.

© 2011-24 National University of Singapore. All Rights Reserved.

Setup a Monitoring Schedule

- The above code sets up an **hourly monitoring job** on your deployed scikit-learn model endpoint.

- Each hour, SageMaker runs a job on an ml.m5.large instance to pull captured input/output data

- Generate a report and save it in S3.

© 2011-24 National University of Singapore. All Rights Reserved.